

EXERCISES

- In Python, code is compiled and ran via a single command. In Java, there is a command which compiles code using the `javac --filename.java` command. This creates a `class` file with the same name as a binary, and then needs to be ran separately using `java --filename--`.
- Calling a function is simple in java. The main concept is that you construct the object FIRST. Then you use various methods that you can use which are inherited by that object (whether it is fathered or grandfathered in via `super`). This is largely the same concept in Python.

```
r = new Rectangle(7,5, new Color(120,170,150)); // Create the object
printAreaAndCircumference(r); // Use method from test.java -->
use getArea & getCircumference from Shape.java
printColor(r); // Use method from test.java -->
Use Color(c) given from Rectangle.java
```

- Calling the constructor `circle` without any arguments/parameters in java returns an error! The justification is in the code snippet below:

```
c = new Circle(); // THERE IS NO base constructor like this with default values
in Circle.java. So it fails!
c = new Circle(int radius) // a constructor with the radius of a circle. THIS
EXISTS in Circle.java
c = new Circle(int radius, color circleColor) // a constructor with the radius AND
color of a circle. THIS EXISTS in Circle.java
```

NOTES

- Some collections and data structures have **defined** lengths. In other words, there are `dynamic` and `defined` / `static` collections. Here's an example:

```
int[] numbers = new int[5]; // this means only five elements can be stored into
this array.
```

Whereas a dynamic one looks like this:

```
import java.util.ArrayList;
ArrayList<Integer> numbers = new ArrayList<>(); //You can store as much as
reasonably needed.
```

- Recursion is when a function or method calls itself until it reaches it's base case(s)! The main idea is that it turns the problem into as many sub-problems as needed until it reaches a solution. The primary

problem with this approach is that the solution seems abstract and it is often times a heavy overhead solution. It works very well for Trees-type algorithmic solutions, though!

- The slides cover a list of dynamic collections, namely:

Array -- Good for flexible lookups. Jack-of-all-trades, master of none. Stores ready-to-call values from memory. Bad at insertions and deletions in the middle of the array.

Linked List -- Works like a chain. Uses slow, $O(n)$ for lookups (usually). Really good for insertions and deletions in the middle of the array.

Stack -- Works really well at the end of the collection (LIFO)

Queue -- Works really well at the beginning of the collection (FIFO)

Tree -- Parent/Child nodes. 2 or more links. Leaf node has no children. Root node is the first node in the tree. Leaf node has no children.